[2.6] Specification Gaming & Goal Misgeneralisation

David Quarel

ILIAD Intensive

October 15, 2026

Two ways reward goes wrong

We tell an RL agent what we want through a **reward function**. Two distinct failure modes:

Two ways reward goes wrong

We tell an RL agent what we want through a **reward function**. Two distinct failure modes:

1. **Goal misspecification** (reward hacking): the designer specified a reward function that incentivises a behaviour that does not match what the designer intended. The reward was optimised, but the desired goal was not internalised by the agent.

Two ways reward goes wrong

We tell an RL agent what we want through a **reward function**. Two distinct failure modes:

1. **Goal misspecification** (reward hacking): the designer specified a reward function that incentivises a behaviour that does not match what the designer intended. The reward was optimised, but the desired goal was not internalised by the agent.
2. **Goal misgeneralisation**: the reward function matches the intended goal, but the agent learns a goal that is perfectly correlated with the intended goal *on the training distribution*, but which diverges on out-of-distribution environments. The agent remains capable, but capably pursues the wrong thing.

Two ways reward goes wrong

We tell an RL agent what we want through a **reward function**. Two distinct failure modes:

1. **Goal misspecification** (reward hacking): the designer specified a reward function that incentivises a behaviour that does not match what the designer intended. The reward was optimised, but the desired goal was not internalised by the agent.
2. **Goal misgeneralisation**: the reward function matches the intended goal, but the agent learns a goal that is perfectly correlated with the intended goal *on the training distribution*, but which diverges on out-of-distribution environments. The agent remains capable, but capably pursues the wrong thing.

Even with a *perfectly specified* reward, it is possible for the agent to still internalise the wrong goal.

The pottery shop

A grid world: a robot shares the floor with fragile **urns** and piles of **shards**, plus a **bin**. We want it to tidy shards into the bin without smashing urns.

- Actions: move around, `PICKUP`, `PUTDOWN`.
- Walking into an urn breaks it into shards.
- Shards are disposed of by picking them up, and dropping them in the bin.
- Play this environment in your browser: https://iliad-intensive.org/uploads/policy-gradients-misgeneralization/play.html



Reward functions as the interface

The reward hypothesis

"That all of what we mean by goals and purposes can be well thought of as maximization of the expected value of the cumulative sum of a received scalar signal (reward)." – Richard Sutton

Assuming this hypothesis, it should be possible (at least in principle) to encourage any sort of behaviour we want by finding just the right reward function.

Reward functions as the interface

The reward hypothesis

"That all of what we mean by goals and purposes can be well thought of as maximization of the expected value of the cumulative sum of a received scalar signal (reward)." – Richard Sutton

Assuming this hypothesis, it should be possible (at least in principle) to encourage any sort of behaviour we want by finding just the right reward function.

We want to train the agent to pick shards up and carry them to the bin. A first attempt, `reward1`:

1. $+1$ for picking up shards,
2. $+1$ for dropping them in the bin.

What behaviour does this reinforce in practice?

Specification gaming: the agent farms reward

Trained on `reward1`, the agent discovers a loophole:

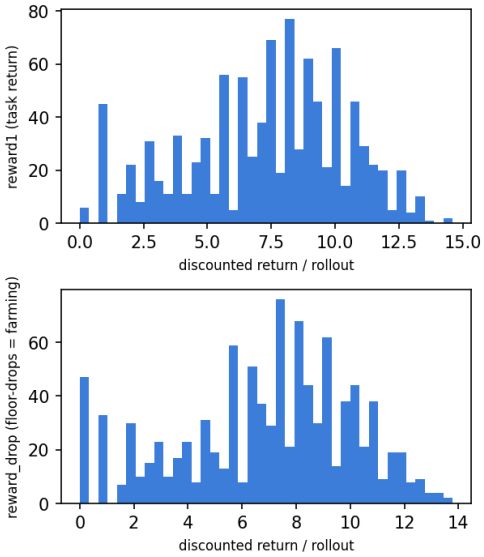
- each `PICKUP` is $+1$,
- dropping on the floor costs nothing,
- so it picks up and drops the same pile over and over.

Specification gaming: the agent farms reward

Trained on `reward1`, the agent discovers a loophole:

- each `PICKUP` is $+1$,
- dropping on the floor costs nothing,
- so it picks up and drops the same pile over and over.

We probe behaviour with extra reward functions: `reward_drop` ($+1$ per floor-drop). Its return tracks `reward1` almost exactly.



Fix 1: potential shaping (the un-hackable hint)

We still want to *hint* that holding shards is good, without creating a loophole.
Removing the reward from picking up shards makes the task very hard to learn.

Potential shaping (Ng, Harada & Russell, 1999) [NHR99]: pick a bounded $\Phi : \mathcal{S} \to \mathbb{R}$ and define a new reward function

$$R^{\Phi}(s, a, s') = R(s, a, s') + \gamma\, \Phi(s') - \Phi(s).$$

Fix 1: potential shaping (the un-hackable hint)

We still want to *hint* that holding shards is good, without creating a loophole.
Removing the reward from picking up shards makes the task very hard to learn.

Potential shaping (Ng, Harada & Russell, 1999) [NHR99]: pick a bounded $\Phi : \mathcal{S} \to \mathbb{R}$ and define a new reward function

$$R^{\Phi}(s, a, s') = R(s, a, s') + \gamma\, \Phi(s') - \Phi(s).$$

Over a trajectory the Φ terms **telescope**: total return changes only by a constant $-\Phi(s_0)$, so the ordering over policies (and the optimum) is unchanged. It is the *only* shaping with this guarantee.

Fix 1: potential shaping (the un-hackable hint)

We still want to *hint* that holding shards is good, without creating a loophole.
Removing the reward from picking up shards makes the task very hard to learn.

Potential shaping (Ng, Harada & Russell, 1999) [NHR99]: pick a bounded $\Phi : \mathcal{S} \to \mathbb{R}$ and define a new reward function

$$R^{\Phi}(s, a, s') = R(s, a, s') + \gamma\,\Phi(s') - \Phi(s).$$

Over a trajectory the Φ terms **telescope**: total return changes only by a constant $-\Phi(s_0)$, so the ordering over policies (and the optimum) is unchanged. It is the *only* shaping with this guarantee.

Here $\Phi(s) = 0.5$ while holding shards. A pickup advances $+\gamma\Phi$; a floor-drop gives it back. A **pickup-then-drop cycle nets exactly zero**: the loophole is closed, while also encouraging the agent to try picking up shards in the first place.

Why shaping preserves the optimum

Write the return from time t with $r_k := R(s_{k-1}, a_{k-1}, s_k)$:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)}\, r_k.$$

Why shaping preserves the optimum

Write the return from time t with $r_k := R(s_{k-1}, a_{k-1}, s_k)$:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)}\, r_k.$$

The shaped reward just adds a potential difference, $r_k^{\Phi} = r_k + \gamma\, \Phi(s_k) - \Phi(s_{k-1})$, so

$$G_t^{\Phi} = \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)}\, r_k^{\Phi} = G_t + \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)} \big(\gamma\, \Phi(s_k) - \Phi(s_{k-1})\big).$$

Why shaping preserves the optimum

Write the return from time t with $r_k := R(s_{k-1}, a_{k-1}, s_k)$:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)}\, r_k.$$

The shaped reward just adds a potential difference, $r_k^{\Phi} = r_k + \gamma\,\Phi(s_k) - \Phi(s_{k-1})$, so

$$G_t^{\Phi} = \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)}\, r_k^{\Phi} = G_t + \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)} \big(\gamma\,\Phi(s_k) - \Phi(s_{k-1})\big).$$

Expanding the shaping sum, consecutive terms **telescope**:

$$\big(\gamma\Phi(s_{t+1}) - \Phi(s_t)\big) + \gamma\big(\gamma\Phi(s_{t+2}) - \Phi(s_{t+1})\big) + \cdots = -\Phi(s_t).$$

Why shaping preserves the optimum

Write the return from time t with $r_k := R(s_{k-1}, a_{k-1}, s_k)$:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)}\, r_k.$$

The shaped reward just adds a potential difference, $r_k^{\Phi} = r_k + \gamma\,\Phi(s_k) - \Phi(s_{k-1})$, so

$$G_t^{\Phi} = \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)}\, r_k^{\Phi} = G_t + \sum_{k=t+1}^{\infty} \gamma^{\,k-(t+1)} \big(\gamma\,\Phi(s_k) - \Phi(s_{k-1})\big).$$

Expanding the shaping sum, consecutive terms **telescope**:

$$\big(\gamma\Phi(s_{t+1}) - \Phi(s_t)\big) + \gamma\big(\gamma\Phi(s_{t+2}) - \Phi(s_{t+1})\big) + \cdots = -\Phi(s_t).$$

Hence $G_t^{\Phi} = G_t - \Phi(s_t)$. At $t = 0$, $G_0^{\Phi} = G_0 - \Phi(s_0)$ differs from G_0 by a constant depending *only on the start state*, not on the policy: so $\arg\max_\pi G_0^{\Phi} = \arg\max_\pi G_0$ and the optimum is unchanged.

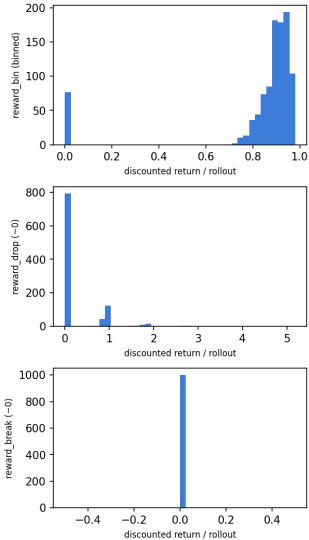
Specification gaming fixed?

Shaping stops farming, but nothing yet discourages breaking urns to manufacture shards.

- Add a -2 penalty for each urn smashed.
- Binning the resulting shard recovers $\approx +1$ (discounted), against a -2 penalty: so breaking-then-cleaning is never worth it.

Retrained on `reward2`, the probes confirm the fix:

- `reward_drop` (farming) ≈ 0,
- `reward_break` (smashing) ≈ 0,
- `reward_bin` > 0: it actually cleans.



Motivating the potential function

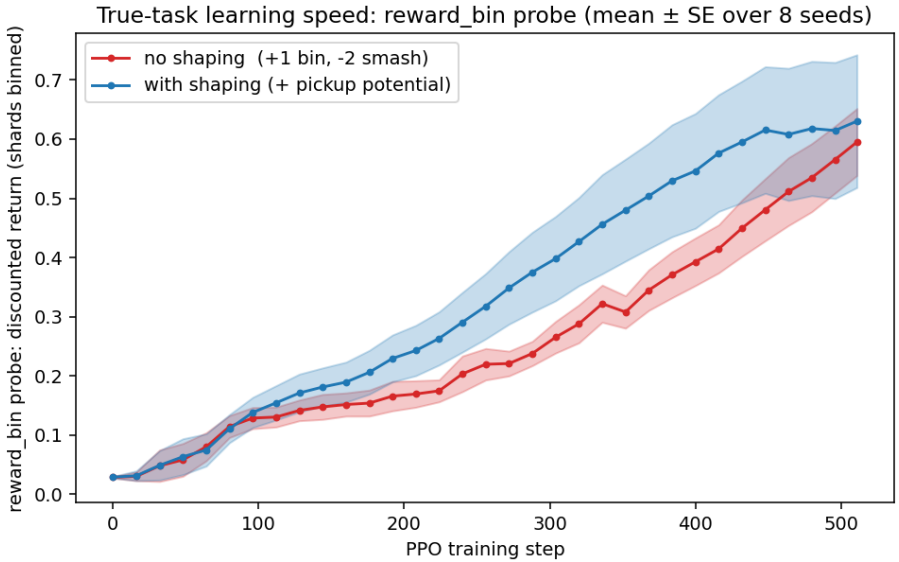
- Why bother with the potential function at all? $+1$ reward for putting shards in the bin, and -2 for smashing urns, also is aligned with the intended goal.

Motivating the potential function

- Why bother with the potential function at all? $+1$ reward for putting shards in the bin, and -2 for smashing urns, also is aligned with the intended goal.
- But the agent still has to learn from experience: the policy does not teleport to whatever maximises the sum of rewards.

Motivating the potential function

- Why bother with the potential function at all? $+1$ reward for putting shards in the bin, and -2 for smashing urns, also is aligned with the intended goal.
- But the agent still has to learn from experience: the policy does not teleport to whatever maximises the sum of rewards.
- Leaving the potential function out robs the agent of useful feedback (that picking up and walking around with shards is instrumentally useful for putting them in the bin).


True-task learning speed: reward_bin probe (mean ± SE over 8 seeds)
no shaping (+1 bin, -2 smash)
with shaping (+ pickup potential)
reward_bin probe: discounted return (shards binned)
PPO training step
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0
100
200
300
400
500

Even perfectly specified reward isn't enough

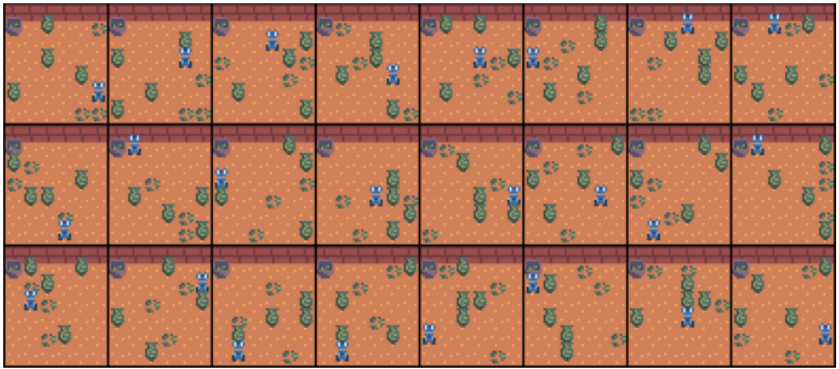
A policy trained on a *single* fixed layout can just memorise a trajectory: it never has to look at where the shards are.

Even perfectly specified reward isn't enough

A policy trained on a *single* fixed layout can just memorise a trajectory: it never has to look at where the shards are.

So we train on **procedurally generated** layouts: random robot and item positions every episode.

As a toy example of insufficent training environment diversity, suppose that the bin is always in the top-left corner.



Goal misgeneralisation: defining it

Langosco et al. (2022) [LKS+22]: an agent trained to maximise R is deployed under distribution shift. *Goal misgeneralisation* occurs if it now gets **low reward** (according to R)because it keeps acting as if it was trained to optimise a *different* $R' \neq R$.

- $R =$ **intended objective** (carry shards to the bin),
- $R' =$ **behavioural objective** (what it actually learned).

Goal misgeneralisation: defining it

Langosco et al. (2022) [LKS+22]: an agent trained to maximise R is deployed under distribution shift. *Goal misgeneralisation* occurs if it now gets **low reward** (according to R)because it keeps acting as if it was trained to optimise a *different* $R' \neq R$.

- $R =$ **intended objective** (carry shards to the bin),
- $R' =$ **behavioural objective** (what it actually learned).

During training, "go to the bin" and "go to the top-left corner" were **indistinguishable**. The simpler one can win, and the model has internalised the wrong goal.

Out of distribution: move the bin

At test time, put the bin somewhere else (e.g. the top-right corner).

Out of distribution: move the bin

At test time, put the bin somewhere else (e.g. the top-right corner). The agent still capably picks up shards and carries them **to the top-left corner**, where it drops them on the floor, ignoring the actual bin.

Out of distribution: move the bin

At test time, put the bin somewhere else (e.g. the top-right corner). The agent still capably picks up shards and carries them **to the top-left corner**, where it drops them on the floor, ignoring the actual bin. Defining a proxy reward function R' that rewards dropping shards at the top left corner, we can probe the agent's behaviour on the shifted layout.



Goal misgeneralisation, measured

On shifted layouts, the agent:

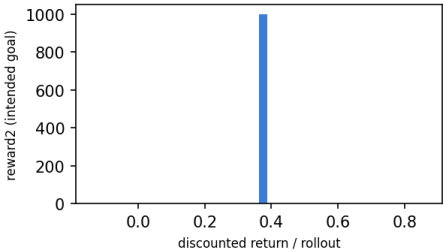
- scores *much lower* on R (putting shards in the bin) than in-distribution,
- but high on R' (dropping shards at the top left corner).

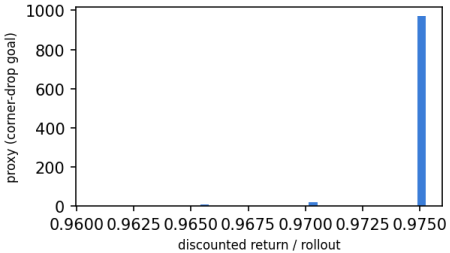
Goal misgeneralisation, measured

On shifted layouts, the agent:

- scores *much lower* on R (putting shards in the bin) than in-distribution,
- but high on R' (dropping shards at the top left corner).

It is still competently optimising a goal, just not what we trained for. There is no black-box way to tell apart an agent that learned R from one that learned R' as the behaviour is the same on in-distribution layouts.





Fixing it: broaden the training distribution

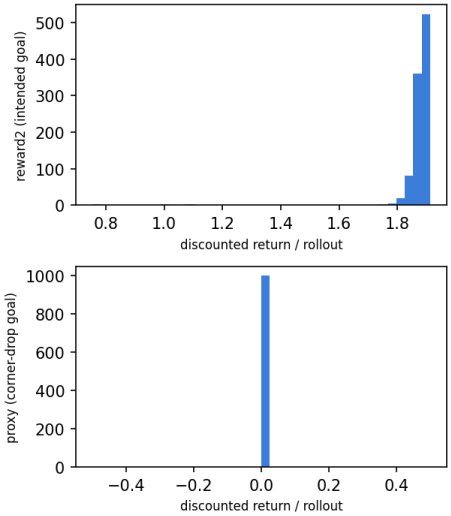
- Randomise the bin position too.
- Now "corner" no longer correlates with "bin", so the agent is forced to learn the intended goal R.

Fixing it: broaden the training distribution

- Randomise the bin position too.
- Now "corner" no longer correlates with "bin", so the agent is forced to learn the intended goal R.
- Result: The training distribution now matches the test distribution, so the agent is no longer misgeneralising.

Fixing it: broaden the training distribution

- Randomise the bin position too.
- Now "corner" no longer correlates with "bin", so the agent is forced to learn the intended goal R.
- Result: The training distribution now matches the test distribution, so the agent is no longer misgeneralising.
- **Caveat:** In practice you often cannot enumerate the shift in advance, so this fix is not always available. Mitigating goal misgeneralisation without it is an open problem.



Summary

- **Specification gaming / outer alignment:** if a behaviour scores higher return than the intended one, the policy will pursue it. Probe with diagnostic rewards; fix with potential shaping (un-hackable) and targeted penalties.

Summary

- **Specification gaming / outer alignment:** if a behaviour scores higher return than the intended one, the policy will pursue it. Probe with diagnostic rewards; fix with potential shaping (un-hackable) and targeted penalties.
- **Goal misgeneralisation / inner alignment:** out of distribution, behaviour is set by the inductive biases, not by what the reward *would* have rewarded. A simpler goal that perfectly correlated during training is what gets pursued when goals diverge.

Summary

- **Specification gaming / outer alignment:** if a behaviour scores higher return than the intended one, the policy will pursue it. Probe with diagnostic rewards; fix with potential shaping (un-hackable) and targeted penalties.
- **Goal misgeneralisation / inner alignment:** out of distribution, behaviour is set by the inductive biases, not by what the reward *would* have rewarded. A simpler goal that perfectly correlated during training is what gets pursued when goals diverge.
- Broadening the training distribution helps, when you can; and "reward-maximising" is necessary but not sufficient for "learned".

References I

[LKS+22] Lauro Langosco, Jack Koch, Lee Sharkey, Jacob Pfau, and David Krueger.
Goal misgeneralization in deep reinforcement learning.
In *International Conference on Machine Learning (ICML)*, 2022.
arXiv:2105.14111.

[NHR99] Andrew Y. Ng, Daishi Harada, and Stuart Russell.
Policy invariance under reward transformations: Theory and application to reward shaping.
In *International Conference on Machine Learning (ICML)*, 1999.

[SVK+22] Rohin Shah, Vikrant Varma, Ramana Kumar, Mary Phuong, Victoria Krakovna, Jonathan Uesato, and Zac Kenton.
Goal misgeneralization: Why correct specifications aren't enough for correct goals.
arXiv preprint arXiv:2210.01790, 2022.